

Deep Learning for Time Series Classification and Regression with aeon

ADBIS 2026 tutorial, Orléans

Ali Ismail-Fawaz¹, Maxime Devanne¹, **Antoine Guillaume**²,
Anthony Bagnall³, **Germain Forestier**^{1,4}

¹IRIMAS, Université de Haute-Alsace ²Novahé

³University of Southampton ⁴Monash University

<https://aeon-tutorials.github.io/ADBIS-2026/>



The next 90 minutes

Time	Part
00:00	Time series machine learning
00:15	The aeon toolkit
00:30	Deep learning architectures for time series
01:00	Hands-on: deep learners in aeon
01:20	Application: rehabilitation assessment, Q&A

What you will take home

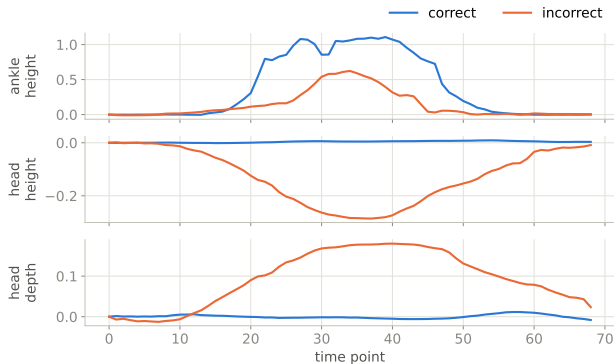
- which architecture to reach for, and why
- how to train, evaluate and inspect one in a few lines
- a notebook that runs on a laptop

All material: <https://github.com/aeon-tutorials/ADBIS-2026>. Prerequisites: Python and basic machine learning.

A time series is an ordered sequence of observations

- **Univariate:** one value per time point.
- **Multivariate:** several *channels* observed together, e.g. the x, y, z coordinates of 22 body joints.
- We learn from a **collection** of series, each with one target.

Order matters: shuffle the time points and the information is gone.



Three of the 66 channels of two recordings of a rehabilitation exercise.

Two supervised tasks on whole series

Time series classification (TSC)

$$f : \mathbb{R}^{d \times m} \rightarrow \{1, \dots, K\}$$

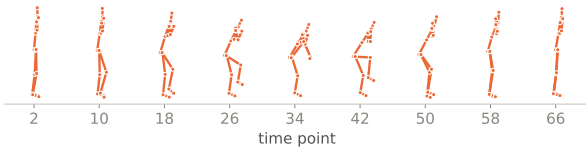
Is this execution of the exercise correct?

d channels, m time points. The target is external to the series: this is not forecasting.

Correct, quality score 0.97



Incorrect, quality score 0.39



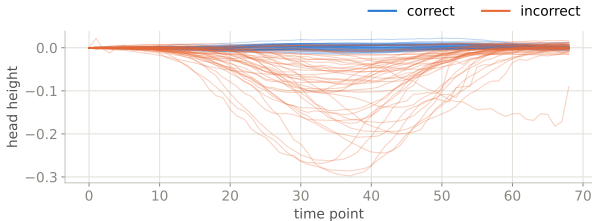
Extrinsic regression (TSER)

$$f : \mathbb{R}^{d \times m} \rightarrow \mathbb{R}$$

What quality score would an expert give?

What makes series hard for standard learners

- The discriminative pattern can be **anywhere** in time.
- It can live at **several scales**: a jerk, a phase, the whole movement.
- It can lie **across channels**, not in any single one.
- Labelled collections are often **small**.



One channel, all training recordings: many incorrect executions look like correct ones.

A rich algorithmic landscape

Distance

1-NN DTW

Feature

Catch22, FreshPRINCE

Interval

DrCIF, QUANT

Shapelet

STC, RDST

Dictionary

BOSS, WEASEL

Convolution

ROCKET, Hydra

Deep learning

InceptionTime, LITE

Hybrid

HIVE-COTE 2

- Each family encodes a different idea of what discriminates series.
- All of them are in `aeon`, behind the same interface. Today: **deep learning**.

Middlehurst, Schäfer, Bagnall. Bake off redux. DMKD 2024

Why deep learning for time series

What it brings

- Features are **learned**, end to end, from raw series.
- **Multivariate** input is native: filters mix channels.
- One backbone, many tasks: classification, regression, clustering, forecasting.
- Scales with data and GPUs; allows explanations such as class activation maps.

What it costs

- Training time and a few sensitive settings.
- Variance between runs, hence **ensembles**.
- On small data, simpler methods can match it: always keep a baseline.

Ismail Fawaz et al. DMKD 2019; Mohammadi Foumani et al. ACM CSUR 2024

aeon: a scikit-learn compatible toolkit for time series

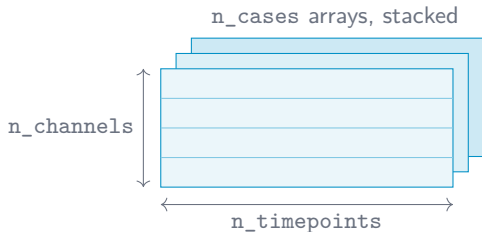
- Open source Python library for **time series machine learning**.
- Learning tasks: classification, regression, clustering, forecasting, anomaly detection, segmentation, similarity search.
- Plus transformations, distances, datasets, benchmarking tools.
- Follows the **scikit-learn** design: estimators, fit, predict.
- State-of-the-art algorithms, maintained by the people who study them.



`pip install aeon` <https://www.aeon-toolkit.org>

Middlehurst et al. aeon: a Python toolkit for learning from time series. JMLR 2024

One data format for collections of series



- A collection is a 3D NumPy array (n_{cases} , n_{channels} , $n_{\text{timepoints}}$).
- Unequal lengths: a list of 2D arrays.
- Targets: a 1D array y .

```
from aeon.datasets import load_classification, load_regression, load_rehab_pile_dataset
```

```
X, y = load_classification("ArrowHead", split="train")      # UCR / UEA archives  
X, y = load_regression("BarCrawl6min", split="train")      # TSER archive  
X, y = load_rehab_pile_dataset("UIPRMD_clf_bn_HS", split="train", fold=0)
```

One interface: construct, fit, predict

```
from aeon.datasets import load_classification
from aeon.classification.deep_learning import FCNClassifier

X_train, y_train = load_classification("ArrowHead", split="train")
X_test, y_test = load_classification("ArrowHead", split="test")

clf = FCNClassifier(n_epochs=50, batch_size=16)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)           # class labels
probas = clf.predict_proba(X_test)    # one probability per class
```

- No tensors, data loaders or training loop: aeon builds, compiles and trains the Keras model.
- The trained network stays accessible as `clf.model_`.

Swap the estimator, keep the code

Regression: change the import

```
from aeon.regression.deep_learning import (  
    ResNetRegressor,  
)  
  
reg = ResNetRegressor(n_epochs=50)  
reg.fit(X_train, y_train)  
y_pred = reg.predict(X_test)
```

A non-deep model: same three lines

```
from aeon.classification.shapelet_based import (  
    RDSTClassifier,  
)  
  
clf = RDSTClassifier()  
clf.fit(X_train, y_train)  
y_pred = clf.predict(X_test)
```

One interface across tasks and algorithm families: the code around the estimator does not change.

scikit-learn tooling applies, 3D input included

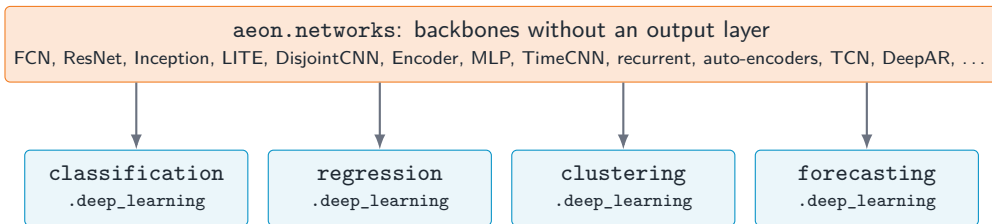
```
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, GridSearchCV
from aeon.transformations.collection import Centerer

# an aeon transformer and a network, chained
pipeline = make_pipeline(Centerer(), FCNClassifier(n_epochs=100))
scores = cross_val_score(pipeline, X, y, cv=5)

search = GridSearchCV(
    FCNClassifier(n_epochs=50),
    param_grid={"n_filters": [[32, 64, 32], [128, 256, 128]]}, # architecture as parameters
    cv=5,
)
search.fit(X, y)
```

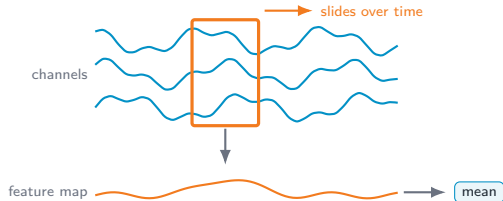
Also: TransformedTargetRegressor, and PredefinedSplit to reuse the folds of an archive.

The deep learning module



- Estimator = network + output layer + the training recipe of the original paper.
- Backend: TensorFlow / Keras. `pip install "aeon[dl]"`
- Shared arguments: `n_epochs`, `batch_size`, `optimizer`, `callbacks`, `save_best_model`, ...

Building blocks: convolution over time, then global average pooling



- A 1D filter spans **all channels** and a short window of time: it detects a local, multi-joint pattern.
- **Weight sharing**: the same detector at every time step, few parameters.
- **Global average pooling (GAP)**: one number per filter, whatever the position of the pattern or the length of the series.
- GAP + linear layer $\Rightarrow$ class activation maps.

Two baselines: MLP and TimeCNN

MLPClassifier

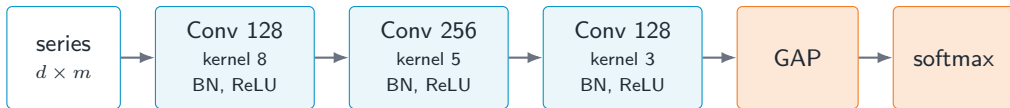
- Flattens the series, three dense layers of 500 units.
- Every weight is tied to one time position: **no notion of time**.
- Large despite its simplicity (2.8M weights on our example).
- Use: sanity check. If it matches your network, time structure is not being exploited.

TimeCNNClassifier

- Two convolutions (kernel 7) with average pooling, sigmoid activations.
- Tiny: a few thousand weights.
- No GAP: a dense layer reads positions.
- Use: the smallest convolutional reference.

Wang et al. IJCNN 2017; Zhao et al. J. Syst. Eng. Electron. 2017

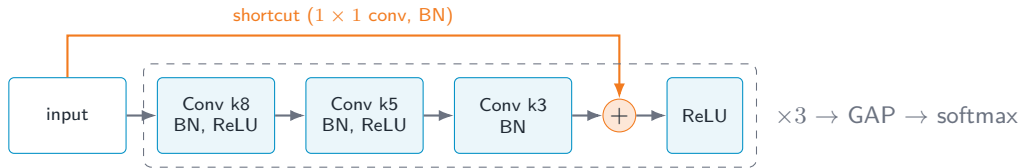
FCN: the strong, simple default



- No pooling between layers: the time resolution is kept to the end.
- About 330k weights on a 66-channel input.
- Fast to train, hard to beat on small problems.
- **Choose it** as first model, and when you need explanations.

Wang, Yan, Oates. Time series classification from scratch with deep neural networks. IJCNN 2017

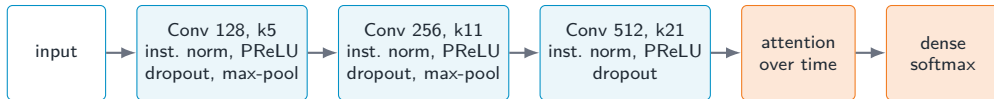
ResNet: depth, made trainable by shortcuts



- Nine convolutions in three residual blocks (64, 128, 128 filters).
- The shortcut lets the gradient bypass a block: depth without the optimisation penalty.
- **Choose it** when you have more data than FCN can use; same GAP ending, same explanations.

Wang, Yan, Oates. IJCNN 2017

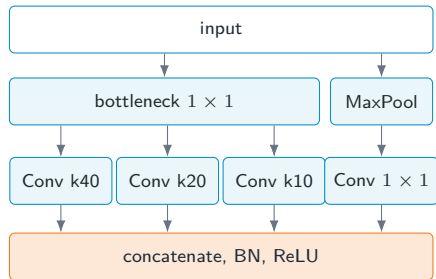
Encoder: convolutions followed by attention



- Half of the last feature maps become softmax weights over time, applied to the other half.
- The network learns *which time region to read*, instead of averaging all of them.
- Wide layers and a dense ending: the largest model of the module (3.3M weights on our example).
- **Choose it** when a short, well-localised event carries the label. It assumes aligned series.

Serrà, Pascual, Karatzoglou. Towards a universal neural network encoder for time series. CCIA 2018

InceptionTime: convolutions at several time scales

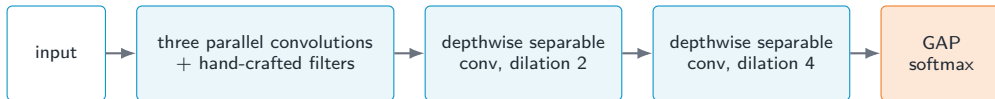


One Inception module

Ismail Fawaz et al. InceptionTime. DMKD 2020; Ismail-Fawaz et al. IEEE BigData 2022

- Filters of length 40, 20 and 10 in parallel: **short and long patterns** in the same layer.
- Six modules, a residual link every three, GAP.
- **InceptionTime** = ensemble of five networks.
- `use_custom_filters=True` gives **H-InceptionTime**: fixed hand-crafted filters (trends, peaks).
- **Choose it** when accuracy comes first.

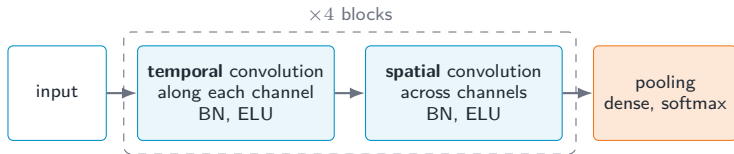
LITE and LITEMV: small by design



- Keeps three ideas of Inception (multiplexed kernels, hand-crafted filters, dilation) and drops the depth.
- **Depthwise separable** convolutions: filter each channel, then mix. Far fewer weights.
- **LITEMV** (`use_litemv=True`) makes the first layer separable too, for multivariate input.
- **LITETime** = ensemble of five. **Choose it** for small data, small memory, embedded inference.

Ismail-Fawaz et al. LITE. DSAA 2023; Look into the LITE. IJDSA 2025

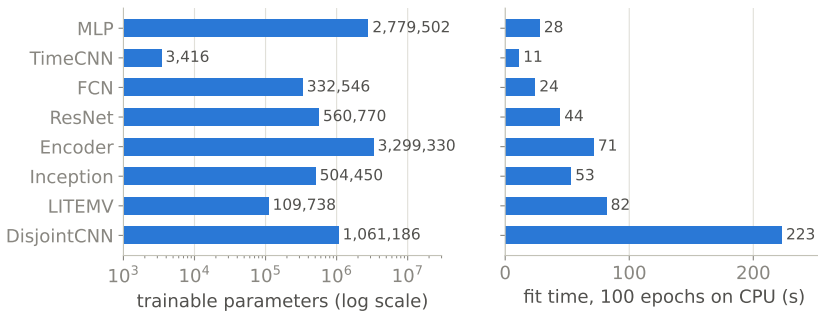
DisjointCNN: convolve over time, then across channels



- A standard 1D convolution mixes time and channels in one step. DisjointCNN **factorises** them (“1+1D”).
- Designed for multivariate series whose channels interact, such as the joints of a body.
- The price is compute: it is the slowest model of the module on CPU.
- **Choose it** for many interacting channels, with a GPU.

Foumani, Tan, Salehi. Disjoint-CNN for multivariate time series classification. ICDMW 2021

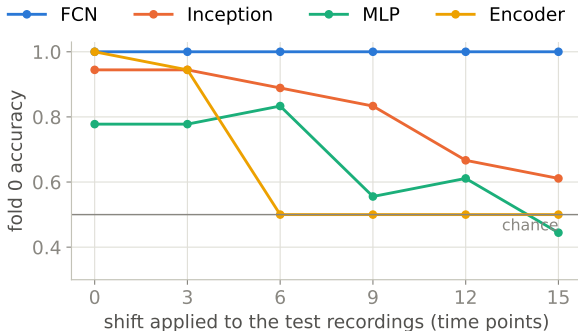
Size and cost, measured on the same problem



- **Size is not depth:** dense layers on a flattened input cost millions of weights.
- LITEMV is the smallest competitive network, but depthwise separable convolutions are slow on CPU.

92 recordings, 66 channels, 69 time points, 100 epochs, batch size 16, single networks, laptop CPU.

The architecture decides what the model is robust to



Test recordings shifted in time, no retraining.

- **FCN** (also ResNet, DisjointCNN): GAP ignores position, predictions do not move.
- **Inception**: filters of 40 points on 69-point series, border effects reach everywhere.
- **MLP, Encoder**: weights tied to positions, they fail once the movement is elsewhere.

Recordings are never perfectly segmented. Inductive bias is a practical matter, not a detail.

Which one should I pick?

If you want...	Start with	Why
a first result, explanations	FCNClassifier	fast, position-invariant, explainable
more capacity, more data	ResNetClassifier	depth with stable training
the best accuracy, compute available	InceptionTimeClassifier	multi-scale filters, ensemble of five
small data, memory or edge devices	LITETimeClassifier	a fraction of the weights
many interacting channels, a GPU	DisjointCNNClassifier	temporal and spatial filters, factorised
a localised event, aligned series	EncoderClassifier	attention over time
a sanity check	MLPClassifier, RDST	no time structure; a non-deep reference

- Every classifier has a **regressor** twin; RecurrentRegressor adds LSTM and GRU.
- **Ensembles** (`n_classifiers`) average out the randomness of initialisation.

Hands-on: follow along

Notebook:

Notebooks/rehab_pile_deep_learning.ipynb

- GitHub: <https://github.com/aeon-tutorials/ADBIS-2026>
- Runs on Google Colab and Binder, links on the tutorial website.
- About 30 minutes on a laptop CPU. Lower N_EPOCHS to go faster.

```
pip install -U "aeon[dl]" matplotlib
```

The plan

1. Load a Rehab-Pile problem
2. Look at the data
3. Train a first network
4. Evaluate it soundly
5. Control and inspect training
6. Regression

Rehab-Pile: 60 assessment problems, one loader

- Existing rehabilitation datasets, unified as skeleton sequences.
- The question is not *which* movement, but **how well** it is performed.
- 39 classification and 21 regression problems.
- Fixed **cross-subject folds**: nobody is in both train and test.
- Names: `UIPRMD_clf_bn_HS` = source, task, exercise.

Source	binary	multi-class	regression
EHE	0	0	6
IRDS	9	0	0
KERAAL	3	3	0
KIMORE	5	0	5
KINECAL	4	0	0
SPHERE	1	0	0
UCDHE	2	2	0
UI-PRMD	10	0	10

Ismail-Fawaz, Devanne, Berretti, Weber, Forestier. arXiv:2507.21018, 2025

Loading a problem

```
from aeon.datasets import load_rehab_pile_dataset

X_train, y_train, meta = load_rehab_pile_dataset(
    "UIPRMD_clf_bn_HS", split="train", fold=0, return_meta=True
)
X_train.shape           # (92, 66, 69): recordings, 22 joints x 3 coordinates, time points
meta["kinematic_tree"]  # chains of joints, to draw the skeleton
```

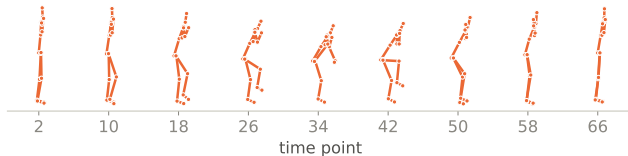
- Running example: the **hurdle step** of UI-PRMD. Ten healthy subjects, correct and incorrect executions.
- Same series, two targets: UIPRMD_clf_bn_HS (label) and UIPRMD_reg_HS (quality score).
- Before training: origin at the pelvis, coordinates divided by 100. **Inputs of order one.**

From a moving skeleton to a multivariate series

Correct, quality score 0.97



Incorrect, quality score 0.39



- Correct: the trunk stays vertical while the leg rises. Incorrect: the body bends, the leg rises less.
- The network never sees a skeleton, only 66 curves. The skeleton is for us.

A first deep classifier

```
from aeon.classification.deep_learning import FCNClassifier

fcf = FCNClassifier(n_epochs=100, random_state=0)
fcf.fit(X_train, y_train)

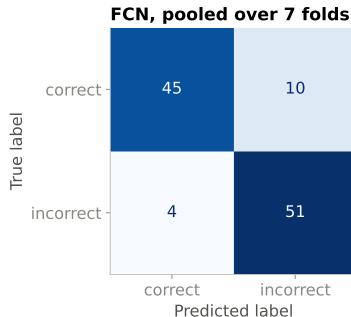
fcf.predict(X_test)           # array([0, 0, 0, ..., 1, 1, 1])
fcf.predict_proba(X_test[:2]) # array([[0.99, 0.01], [1.00, 0.00]])
```

- Behind `fit`: Keras layout, one-hot labels, Adam, learning rate reduction on plateau, and the weights of the best epoch are kept.
- Fold 0: 18 test recordings, all correctly classified. **Too good to trust?**

Evaluate over all folds

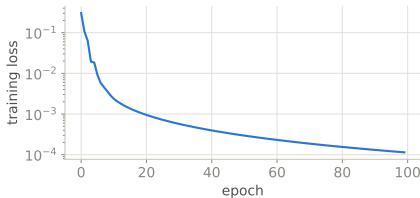
```
def evaluate(make_estimator, problem, n_folds):  
    y_true, y_pred = [], []  
    for fold in range(n_folds):  
        X_tr, y_tr, X_te, y_te = load_fold(problem, fold)  
        estimator = make_estimator().fit(X_tr, y_tr)  
        y_true.append(y_te)  
        y_pred.append(estimator.predict(X_te))  
    return np.concatenate(y_true), np.concatenate(y_pred)
```

- The 7 test sets are disjoint and cover all 110 recordings: **pool** the predictions.
- Fold 0: accuracy 1.00. All folds: **0.87**.



Controlling and inspecting training

```
fcf = FCNClassifier(  
    n_epochs=100, batch_size=8,  
    optimizer=tf.keras.optimizers.Adam(5e-4),  
    callbacks=[tf.keras.callbacks.EarlyStopping(  
        monitor="loss", patience=30)],  
    save_best_model=True, file_path="models/",  
    best_file_name="fcf_hurdle_step",  
)  
fcf.fit(X_train, y_train)  
history = fcf.summary()           # losses and metrics  
  
new = FCNClassifier()             # later, no training  
new.load_model("models/fcf_hurdle_step.keras",  
               classes=fcf.classes_)
```



Any Keras optimizer, loss, metric or callback. `aeon` monitors the *training* loss: no validation data is held out.

Two practical lessons for small training budgets

1. Scale the inputs

- Raw coordinates were in the hundreds, and depended on where the subject stood.
- We centre on the pelvis and divide by 100: the skeleton geometry is preserved.
- Without it, MLP and TimeCNN predicted a single class.

2. Mind the batch size

- Defaults follow the papers: 1500 to 2000 epochs, batches of 16 to 64.
- 92 recordings, 100 epochs, batches of 64: about 200 gradient steps.
- Too few for **batch normalisation** statistics to settle: ResNet and Inception predicted a single class. Batches of 16 fixed it.

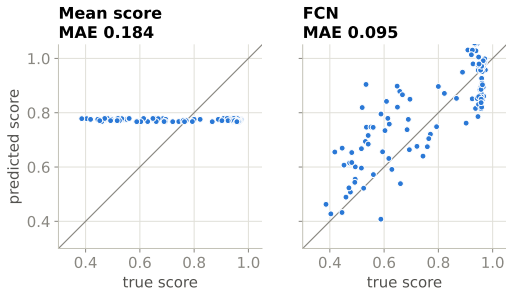
A network stuck at chance level after a short training is often a preprocessing or budget problem, not an architecture problem.

Regression: only the import changes

```
from aeon.regression import DummyRegressor
from aeon.regression.deep_learning import (
    FCNRegressor,
)

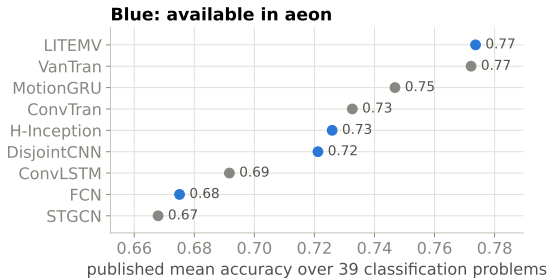
evaluate(lambda: DummyRegressor(),
         "UIPRMD_reg_HS", n_folds=7)
evaluate(lambda: FCNRegressor(n_epochs=100),
         "UIPRMD_reg_HS", n_folds=7)
```

Mean absolute error, pooled over 7 folds: mean score **0.184**, FCN **0.095** (correlation 0.82).



Every deep classifier has a regressor twin. Report the trivial baseline next to it: here the mean score, a flat line.

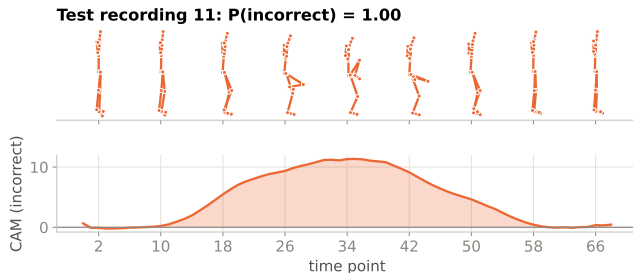
The published benchmark answers “which model is best?”



- 60 problems, 1500 epochs, five runs, ensembles: a question for a benchmark, not for a laptop.
- Results are public; the notebook reads them.
- **LITEMV**, the lightest model, is among the best on skeleton data.
- On our hurdle step problem: FCN 0.81, DisjointCNN 0.90, H-Inception 0.93, LITEMV 0.97.

Results: <https://github.com/MSD-IRIMAS/DeepRehabPile>

When does the model decide? Class activation maps

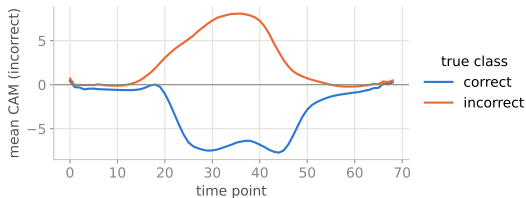


$$\text{CAM}_c(t) = \sum_k w_k^c A_k(t)$$

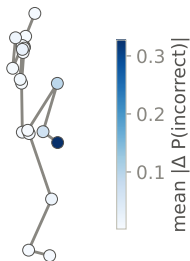
- A_k : last feature maps; w^c : weights of class c after GAP.
- A few lines on `fcn.model_`.
- Evidence for “incorrect” builds up **while the leg is in the air.**

Zhou et al. Learning deep features for discriminative localization. CVPR 2016

Where on the body? Occlusion, for any estimator



Mean CAM over the test recordings of each class.



- Freeze one joint at its mean position, predict again, measure the change.
- Model-agnostic: works with any estimator, deep or not.
- The decision rests on the **lifted leg**, its foot first.

Both views point to where the two executions differ: a sanity check before trusting a model.

Beyond the estimators: build on `aeon.networks`

```
from aeon.networks import FCNNNetwork

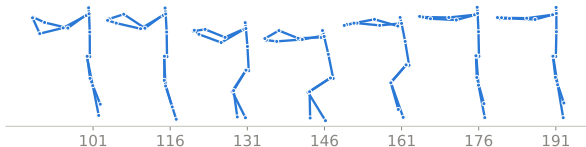
inputs, features = FCNNNetwork().build_network(
    input_shape=(n_timepoints, n_channels))
incorrect = tf.keras.layers.Dense(
    1, activation="sigmoid", name="incorrect")(features)
score = tf.keras.layers.Dense(
    1, activation="sigmoid", name="score")(features)

model = tf.keras.Model(inputs, [incorrect, score])
model.compile(optimizer="adam", loss={
    "incorrect": "binary_crossentropy", "score": "mse"})
```

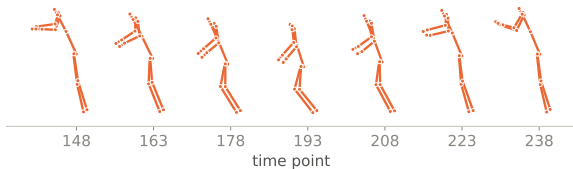
- A network returns the Keras input and its last hidden layer: **the output is yours.**
- Here: one backbone, two heads, label and score learned together.
- From there on it is plain Keras.

The same code on clinical data: KIMORE

Expert score 100 / 100



Expert score 7 / 100



- 71 people, 31 of them patients; expert score out of 100; 18 joints, 557 time points.
- Only the problem name and the kinematic tree change. Test folds hold **patients only**.

Model	MAE
RDST (non-deep baseline)	17.0
FCN (100 epochs)	27.4

Small clinical datasets can be too small for deep learning, in the published benchmark too. This is where a non-deep baseline earns its place.

Take-aways

In short

- **One interface.** A deep network in `aeon` is an estimator; classification and regression differ by an import; baselines are one line away.
- **Architectures are choices:** position invariance, time scales, channels, size, cost.
- **Evaluate over all folds**, next to a baseline.
- **Nothing is hidden:** Keras arguments, `model_`, `aeon.networks`.

Resources

- aeon-toolkit.org
- github.com/aeon-toolkit/aeon
- aeon-tutorials.github.io/ADBIS-2026
- Discord: link on the website

Supported by the P16 Inria grant, ANR DELEGATION (ANR-21-CE23-0014) and ANR Terminus (ANR-25-CE23-2879).

Thank you. Questions?